Демо 2. Headless-режим и GitHub Actions

Что делаем

Запускаем Claude в headless-режиме через `claude -p` — без интерактивной оболочки. Разбираем ключевые флаги: `--allowedTools`, `--output-format`, `--max-turns`, `--resume`, `--bare`. Настраиваем GitHub Action `anthropics/claude-code-action@v1` для реакции на `@claude` в PR. Реализуем паттерн Security Gate как шаг pre-deploy.

К концу демо вы умеете запускать Claude из скриптов, сохранять JSON-результаты, продолжать сессии по session_id и встраивать Claude в CI/CD.

Подготовка

1. Claude Code установлен: `claude doctor` проходит.
2. Переменная `ANTHROPIC_API_KEY` установлена в shell-профиле.
3. Репозиторий на GitHub с доступом `contents: write` и `pull-requests: write`.
4. Утилита `jq` для парсинга JSON. Установка: `brew install jq` или `apt install jq`.
5. Любой проект с `.env.example` и Git-историей минимум на 2 коммита.

Шаг 1. Базовый вызов

В терминале:

```
claude -p "Посмотри структуру текущего проекта и напиши один абзац описания."
```

Claude запускается, читает файлы, возвращает текст в stdout. Сессия закрывается.

Добавим флаг `--bare` для CI-стабильности:

```
claude --bare -p "Посмотри структуру и напиши абзац описания." \
  --allowedTools "Read,Glob,Grep"
```

`--bare` пропускает autodiscovery: хуки, навыки, MCP, CLAUDE.md не загружаются. Результат одинаков на любой машине. Без `--allowedTools` скрипт повиснет на первой попытке чтения.

Шаг 2. JSON-вывод и jq

Запрос с `--output-format json`:

```
claude -p "Сколько строк в package.json?" \
  --output-format json \
  --allowedTools "Read" \
  --bare
```

Ответ — JSON-объект с полями `result`, `session_id`, usage-метриками, и `is_error`.

Вытащим только текст:

```
claude -p "Сколько строк в package.json?" \
  --output-format json \
  --allowedTools "Read" \
  --bare | jq -r '.result'
```

Сохраним в файл:

```
claude -p "Опиши проект" --output-format json --bare \
  --allowedTools "Read,Glob,Grep" \
  | jq -r '.result' > description.md
```

Шаг 3. Структурированный вывод по JSON Schema

Вариант строгого формата — `--json-schema`:

```
claude -p "Извлеки имена всех функций из src/utils.ts" \
  --output-format json \
  --json-schema '{"type":"object","properties":{"functions":
{"type":"array","items":{"type":"string"}},"required":["functions"]}}' \
  --allowedTools "Read" \
  --bare \
  | jq '.structured_output'
```

Claude вернёт только JSON, соответствующий схеме. Удобно для скриптов, которые парсят результат и передают дальше.

Шаг 4. Продолжение сессии через session_id

Первый вызов — захватываем `session_id`:

```
session_id=$(claude -p "Прочитай src/api/ и определи основные паттерны" \
  --output-format json \
  --allowedTools "Read,Glob,Grep" \
  --bare \
  | jq -r '.session_id')

echo "Session ID: $session_id"
```

Второй вызов — возобновляем с контекстом:

```
claude -p "Предложи три улучшения для error handling" \
  --resume "$session_id" \
  --output-format json \
  --bare \
  | jq -r '.result'
```

Третий — сохраняем отчёт:

```
claude -p "Сделай итоговый отчёт" \
  --resume "$session_id" \
  --output-format json \
  --bare \
  | jq -r '.result' > report.md
```

Claude помнит весь контекст: те же файлы, те же находки.

Шаг 5. Передача вывода команды

Передача stdout команды как входа:

```
gh pr diff 42 | claude -p \
  --append-system-prompt "You are a security engineer. Review for
vulnerabilities." \
  --output-format json \
  --bare \
  --allowedTools "Read" \
  | jq -r '.result' > pr-42-security-review.md
```

Типовой паттерн для ревью PR в скриптах.

Шаг 6. Shell-скрипт ревью staged-изменений

Создадим файл `scripts/review-staged.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

DIFF=$(git diff --cached)

if [ -z "$DIFF" ]; then
  echo "Нет staged-изменений"
  exit 0
fi
```

```
echo "$DIFF" | claude -p \
  "Ревью диффа: баги, безопасность, стиль. Markdown-резюме." \
  --output-format json \
  --bare \
  --allowedTools "Read" \
  | jq -r '.result' > .last-review.md

echo "Ревью сохранено в .last-review.md"
```

Сделать исполняемым:

```
chmod +x scripts/review-staged.sh
```

Запуск:

```
git add .
./scripts/review-staged.sh
cat .last-review.md
```

Шаг 7. GitHub Action — установка

В интерактивном Claude Code:

```
/install-github-app
```

Команда:

1. Откроет OAuth-поток установки app на ваши репозитории.
2. Добавит `ANTHROPIC_API_KEY` в secrets репозитория.
3. Создаст PR с двумя workflow-файлами: `.github/workflows/claude.yml` и `.github/workflows/claude-code-review.yml`.

Мёржим PR. Теперь:

- Комментарий с `@claude ...` в любой PR/issue — Claude берёт задачу.
- Новый PR — автоматический код-ревью от Claude.

Шаг 8. Кастомизация workflow

Откроем `.github/workflows/claude-code-review.yml`. Добавим параметры экономии:

```yaml
- uses: anthropics/claude-code-action@v1
  with:
    prompt: |
      Review this PR for:
      1. Security vulnerabilities
      2. Performance issues
      3. Missing error handling
      4. Breaking changes
      Be concise. Flag only real issues, not style preferences.
    anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
    claude_args: |
      --max-turns 5
      --model claude-sonnet-4-6
      --append-system-prompt "Focus on critical issues only."
```

`claude_args` принимает все CLI-опции. `--max-turns 5` защищает от перерасхода. `claude-sonnet-4-6` — дешёвая модель для рутинного ревью.

Важно: в v1 GA старые поля вроде `allowed_tools`, `custom_instructions`, `model` заменены на `claude_args`.

Шаг 9. Паттерн Security Gate

Создадим pre-deploy workflow `.github/workflows/security-gate.yml`:

```yaml
name: Security Gate

on:
  push:
    branches: [main]

jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
        with:
          fetch-depth: 2
      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code
      - name: Run security check
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          RESULT=$(claude -p "Проанализируй git diff HEAD~1..HEAD.
          Проверь: секреты, SQL-инъекции, невалидированный ввод.
          Первая строка — PASS или FAIL." \
            --allowedTools "Bash(git diff *),Bash(git log *),Read" \
            --max-turns 5 \
            --bare \
            --output-format json | jq -r '.result')

          echo "$RESULT"

          if echo "$RESULT" | head -1 | grep -q "^FAIL"; then
            echo "Security gate failed"
            exit 1
          fi
```

Деплой идёт дальше только если первая строка ответа — `PASS`. Базовая защита от случайных утечек.

Шаг 10. Контроль расходов

Обязательные настройки в CI:

```
jobs:
  job-name:
    timeout-minutes: 10        # лимит на job
    steps:
      - uses: anthropics/claude-code-action@v1
        with:
          claude_args: |
            --max-turns 5
            --model claude-sonnet-4-6
```

Правила:

- `--max-turns` — всегда. Иначе при сбое Claude потратит весь бюджет.
- Sonnet 4.6 для рутины, Opus 4.7 только где нужны рассуждения.
- `timeout-minutes: 10` — страховка от зависаний.
- Тестируйте локально: `ANTHROPIC_API_KEY=... claude -p "..." --bare`.

Шаг 11. Запрет на секреты в `.claude/settings.json`

Добавляем deny-список:

```
{
  "permissions": {
    "deny": [
      "Read(.env*)",
      "Read(**/credentials*)",
      "Read(**/*secret*)"
    ]
  }
}
```

Claude не сможет прочитать `.env`, любые файлы с `credentials` или `secret` в пути. Даже если попросите в промпте, он заблокирует чтение.

Чеклист

- [ ] Базовый `claude -p "..."` вернул ответ в терминал.
- [ ] `--bare` добавлен — autodiscovery пропускается.
- [ ] `--output-format json | jq -r '.result'` вытаскивает только текст.
- [ ] Сохранили `session_id` из первого вызова, продолжили второй через `--resume`.
- [ ] Создали shell-скрипт `scripts/review-staged.sh`, запустили на staged-изменениях.
- [ ] Запустили `/install-github-app`, получили PR с workflow-файлами.
- [ ] Настроили `claude_args` в workflow — `--max-turns 5`, `--model claude-sonnet-4-6`.
- [ ] Создали `.github/workflows/security-gate.yml` с PASS/FAIL-логикой.
- [ ] Добавили `timeout-minutes: 10` на критичные job.

- [ ] Закрыли доступ к секретам через deny-список в `settings.json`.